XSS Payload Generator

Project Report — Academic Cybersecurity Simulation

Name: Dharshana S
Roll No: 727823TUCY009
Sri Krishna College of Technology

---

1. Introduction

Cross-Site Scripting (XSS) is a common web application vulnerability that allows attackers to inject malicious scripts into web pages viewed by users. These scripts can execute in the victim's browser, leading to data theft, session hijacking, or unauthorized actions.

This project implements a safe and controlled simulation of XSS attacks using a custom-built XSS Payload Generator tool. The tool generates various payloads that are used to test vulnerable web applications in a lab environment. The testing was performed using Kali Linux as the attacker machine and Metasploitable2 as the target system.

The objective of this project is to understand how XSS attacks work and to demonstrate different payload techniques in a controlled and ethical environment.

---

2. Tool Description

The XSS Payload Generator is a Python-based tool that generates multiple types of XSS payloads automatically. These payloads can be used to test input fields in vulnerable web applications.

The tool produces different categories of payloads such as:

- Script-based payloads
- Event-based payloads
- HTML injection payloads

Each payload is printed along with the roll number and timestamp for tracking purposes.

Example Payloads Generated:

- <script>alert('XSS')</script>
- <img src=x onerror=alert('XSS')>
- <svg onload=alert('XSS')>
- '><script>alert(1)</script>
- <body onload=alert('XSS')>

Technologies Used:

- Python 3

- Kali Linux
- Metasploitable2 (DVWA)

---

3. System Architecture

The project is divided into the following components:

Component	File	Description
Core Tool	tool_main.py	Generates XSS payloads
Setup Script	setup_lab.py	Initializes lab environment
Execution Script	run_tool.py	Runs payload generator
Analysis Script	analyze_results.py	Displays observations
Target System	Metasploitable2 (DVWA)	Vulnerable web app

Workflow:

1. Tool generates payloads
2. Payloads are injected into DVWA
3. Browser executes scripts
4. Results are observed and recorded

---

4. Test Cases & Results

Test Case	Description	Payload Used	Result
TC-1	Basic script injection	<script>alert(1)</script>	Alert executed successfully
TC-2	Event-based XSS	<img src=x onerror=alert(1)>	Alert executed successfully
TC-3	Input breaking injection	'><script>alert(1)</script>	Alert executed successfully

---

```
┌──(kali㉿kali)-[~/SKCT_727823TUCY009_XSS_Payload_Generator/code]
└─$ nano tool_main.py

┌──(kali㉿kali)-[~/SKCT_727823TUCY009_XSS_Payload_Generator/code]
└─$ python3 tool_main.py

Generated Payloads:
1. <script>alert('XSS')</script>
2. <img src=x onerror=alert('XSS')>
3. <svg onload=alert('XSS')>
4. '><script>alert(1)</script>
5. <body onload=alert('XSS')>
Roll No: 727823TUCY009
Timestamp: 2026-03-29 13:32:06.853589

┌──(kali㉿kali)-[~/SKCT_727823TUCY009_XSS_Payload_Generator/code]
└─$ firefox http://192.168.1.5
```

Figure 1: Execution of XSS Payload Generator showing generated payloads with timestamp

Test Case Demonstrations:

- TC-1 shows basic XSS execution

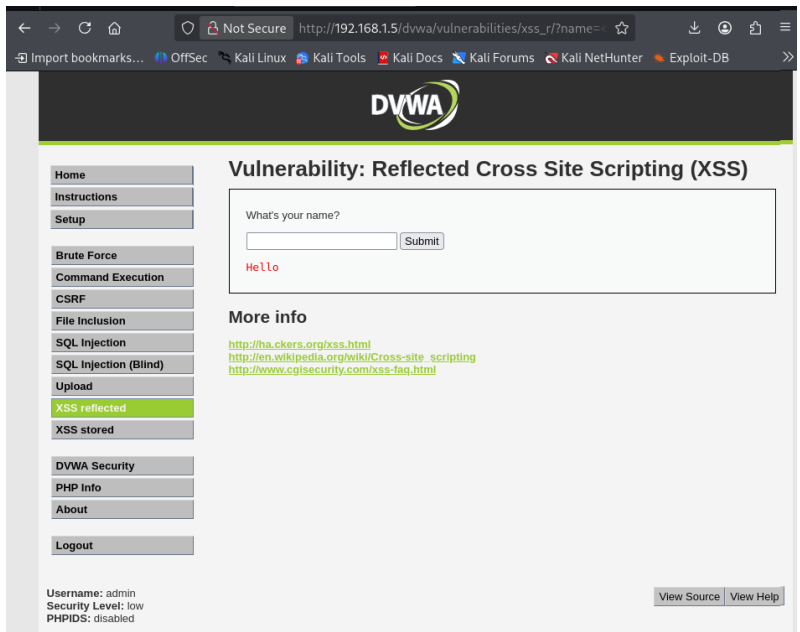

Figure 2: Basic XSS payload execution showing alert popup

- TC-2 demonstrates bypass technique using events

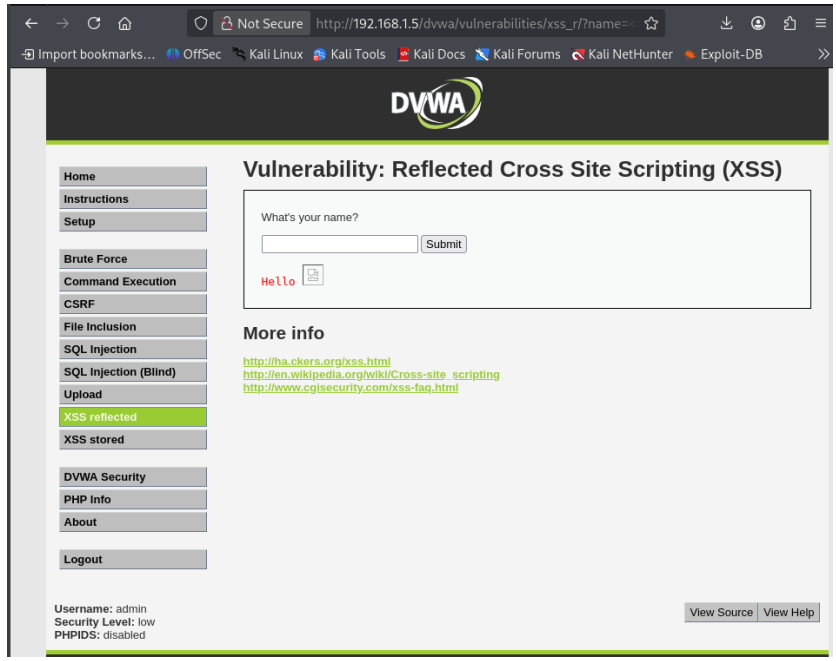

Figure 3: Event-based XSS payload using image onerror

- TC-3 shows breaking of HTML structure for injection

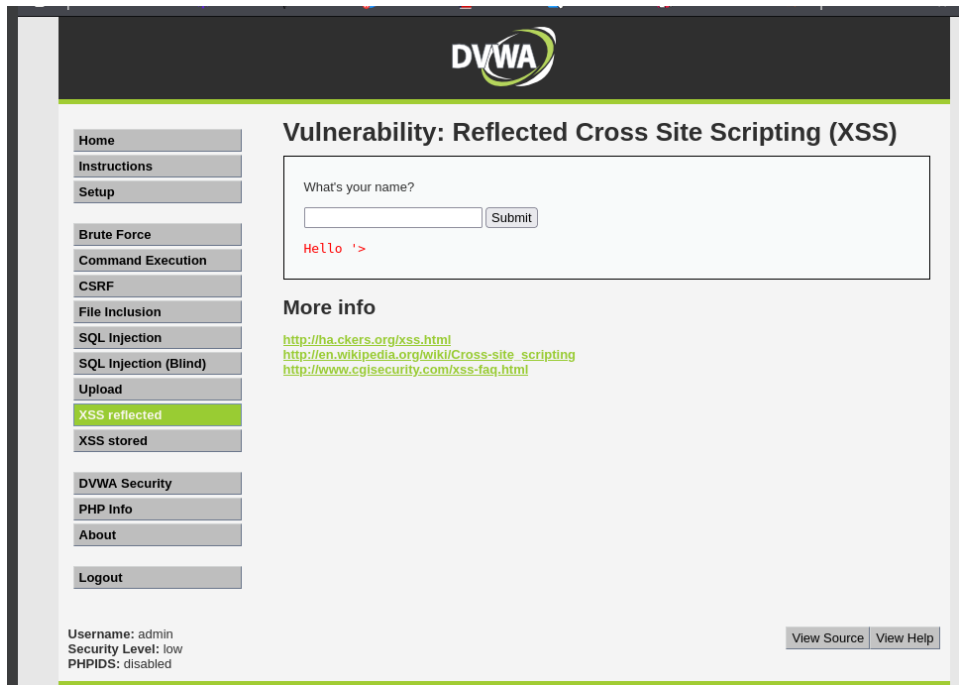

Figure 4: HTML injection payload execution

5. Analysis of Results

The test cases clearly demonstrate that web applications with low security settings are highly vulnerable to XSS attacks.

- All payloads executed successfully in DVWA (Low security mode)
- Event-based payloads worked even when script filtering is applied
- Input-breaking payloads showed how attackers can bypass HTML structures

These results highlight the importance of:

- Input validation
- Output encoding
- Secure coding practices

The tool successfully generated diverse payloads and proved effective for testing XSS vulnerabilities.

---

6. Real Error Encountered

Error 1:

IndentationError: expected an indented block after function definition

Cause:

Improper indentation in Python after defining functions.

Fix:

Added proper indentation (4 spaces) inside functions.

Error 2:

NameError: name 'name' is not defined

Cause:

Incorrect usage of name instead of __name__.

Fix:

Replaced with:

if __name__ == "__main__":

Error 3:

SyntaxError: 'return' outside function

Cause:

Return statement placed outside function block.

Fix:

Adjusted indentation so return is inside the function.

---

7. Ethical Considerations

This project was conducted strictly within a controlled lab environment.

- Only Metasploitable2 was used as the target
- No real-world systems were tested
- No real user data was used
- All activities were performed for educational purposes

The project follows ethical hacking principles and aims to improve understanding of web security vulnerabilities.